

Análisis de Series de Tiempo II

Transformers para Series Temporales

MSc. Fernando Silva

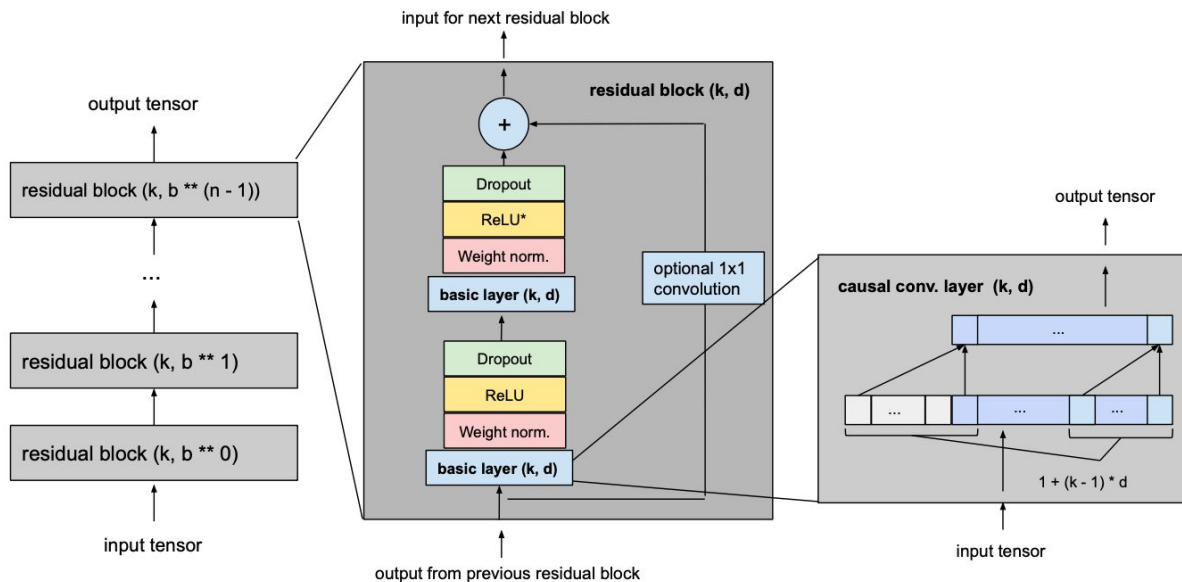


Agenda

- Repaso
- Positional Encoding y Temporal Embeddings
- Embeddings de Series Temporales
- Transformers para Forecasting
- Alternativas al Attention

Repaso

¿Qué es una TCN?



Alternativa genérica a RNN/LSTM para modelado de secuencias

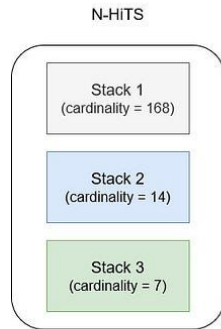
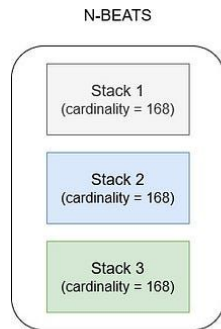
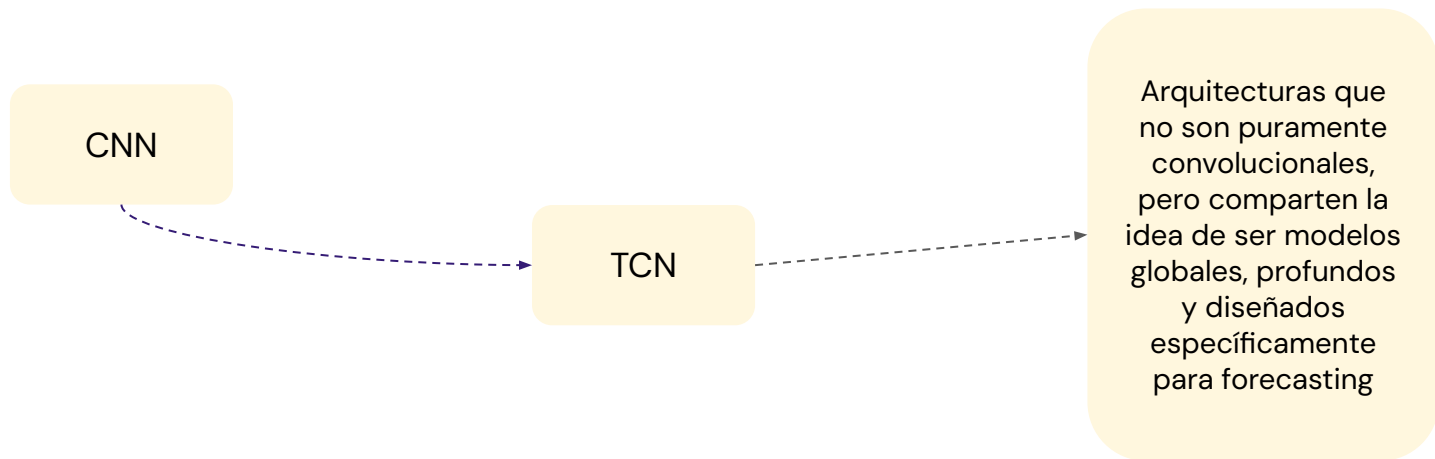
Componentes: convoluciones causales + dilatadas + conexiones residuales + normalización de pesos + dropout

Produce una secuencia de salida de la misma longitud que la entrada (arquitectura "seq2seq")

Arquitecturas basadas en descomposición

¿Cómo se implementa esta idea de descomposición dentro de una red profunda moderna?

N-BEATS y N-HiTS, dos arquitecturas que llevan este principio al extremo mediante "basis expansion"



Positional Encoding y Temporal Embeddings

Positional Encoding

- En el Transformer el orden debe inyectarse como dato adicional en la representación de entrada.
- El PE sinusoidal codifica el índice dentro de la ventana, no el tiempo calendario. Dos ventanas idénticas en forma, una centrada en un martes de agosto y otra en un domingo de diciembre, reciben exactamente la misma señal posicional.
- En una serie, lo que importa casi siempre es el desfase $t-s$ (un retraso de 24 horas es un retraso de 24 horas en cualquier punto del año), no la posición absoluta dentro del recorte arbitrario que define la ventana. Codificar posición absoluta obliga al modelo a aprender la invarianza por traslación que podríamos haberle dado.
- El forecasting tiene una estructura que el NLP no tiene: cada posición viene con metadatos temporales conocidos a futuro (hora del día, día de la semana, día del mes, mes, semana del año, festivo, trimestre fiscal).

Temporal Embeddings

- **Estrategia 1, Embeddings categóricos por granularidad**

Una tabla de embeddings independiente por atributo (24 entradas para hora, 7 para día de semana, 12 para mes, 32 para día del mes) y se suman todos.

- **Estrategia 2, Features continuas normalizadas**

Cada atributo se mapea a $[-0.5, 0.5]$ y se proyecta linealmente. Más económico y sin tabla, pierde la capacidad de representar relaciones no monótonas entre categorías (que diciembre y enero sean vecinos por ejemplo).

- **Estrategia 3, Time2Vec**

Aprende simultáneamente el término no periódico (tendencia lineal) y un banco de periodicidades cuyas frecuencias son parámetros entrenables.

Embeddings de Series Temporales

Embeddings de Series Temporales

¿Qué es un token en una serie de tiempo?

Opción 1, Un punto = un token (Informer, Autoformer). Longitud de secuencia L igual al número de observaciones. Expresa dependencias entre instantes

Opción 2, Un parche = un token (PatchTST). Se agrupan P observaciones contiguas con paso S y se proyectan linealmente a d_{model} . Expresa dependencias entre subsecuencias.

Opción 3, Una serie completa = un token (iTransformer). Se invierte la dimensión: cada variable pasa a ser un token cuyo embedding es toda su historia, y la atención opera entre variables, no entre instantes. La dependencia temporal queda a cargo de la red feed-forward. Expresa dependencias cruzadas.

Embeddings de Covariables

Un pipeline completo de forecasting con Transformers combina cuatro familias de entrada, y cada una entra por una vía distinta:

- **Objetivo pasado:** proyección del valor (densa o conv-1D), o parches.
- **Covariables conocidas a futuro:** calendario, festivos, promociones anunciadas, etc.
- **Covariables observadas:** sólo disponibles hasta t .
- **Covariables estáticas:** identidad de serie, categoría de producto, región, etc.

Temporal Fusion Transformer es la arquitectura que formaliza esta separación: variable selection networks por familia que aprenden pesos de importancia por variable y por instante.

Transformers para Forecasting

Vanilla Transformer para Series de Tiempo

- El Vanilla Transformer no puede entrar al régimen de horizonte largo. Con $L=1000$ la matriz de atención tiene 10^6 entradas por cabeza y por capa, la memoria explota antes de poder aprovechar el contexto.
- Además el decoder autorregresivo hace inferencia paso a paso y acumula error.

¿Necesitamos calcular las L^2 interacciones?

No. El mapa de atención es disperso (cola larga), basta con las $\sim \log L$ queries dominantes (ProbSparse), se destila la longitud entre capas (distilling) y se genera todo el horizonte de una sola pasada (decoder generativo): **Informer**

Informer

Idea: hacer la atención escalable explotando la dispersión del mapa de atención.

ProbSparse Self-Attention

- Parte de la observación de que no todas las queries contribuyen de la misma manera a la atención. Algunas queries concentran su atención en determinadas posiciones, mientras que otras producen distribuciones cercanas a la uniforme.
- Para identificar las queries más relevantes, se calcula una medida de dispersión.
- ProbSparse mantiene el cálculo detallado para las queries más relevantes y aproxima las restantes, reduciendo significativamente el costo computacional.

Informer

Self-attention Distilling

- El encoder reduce progresivamente la longitud de la secuencia entre sus capas.
- El MaxPool reduce la longitud a la mitad en cada capa. Esto disminuye el uso de memoria y permite conservar principalmente las características más relevantes.

Generative Style Decoder

- El decoder recibe como entrada: [segmento inicial del pasado;ceros para el horizonte futuro]
- Luego, genera todo el horizonte de predicción en una sola pasada (forward pass), en lugar de predecir un paso a la vez.

Reducir la longitud en el encoder y generar todo el futuro de una vez hace que el modelo sea mucho más eficiente para predicción de series temporales de largo horizonte.

Informer

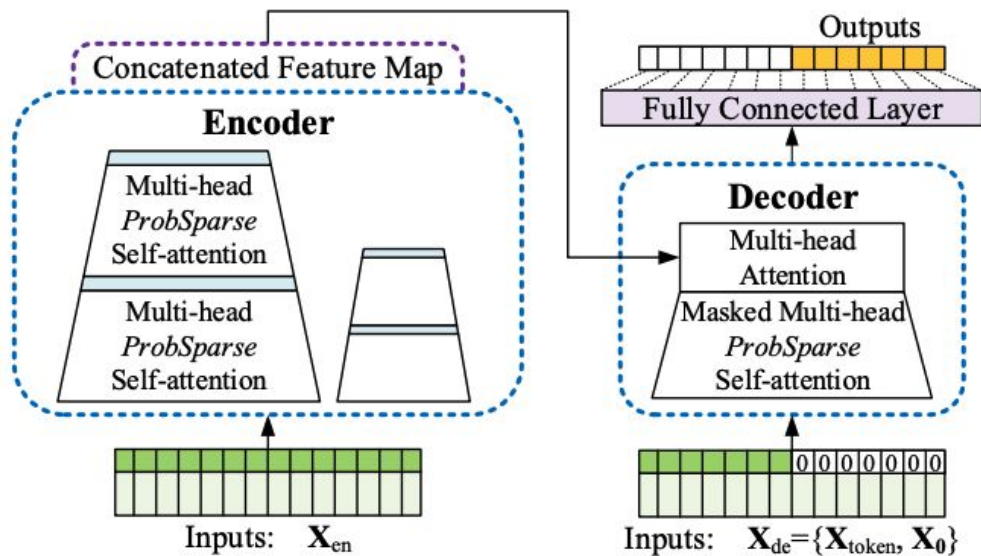


Figure 2: Informer model overview. Left: The encoder receives massive long sequence inputs (green series). We replace canonical self-attention with the proposed *ProbSparse* self-attention. The blue trapezoid is the self-attention distilling operation to extract dominating attention, reducing the network size sharply. The layer stacking replicas increase robustness. Right: The decoder receives long sequence inputs, pads the target elements into zero, measures the weighted attention composition of the feature map, and instantly predicts output elements (orange series) in a generative style.

Informer

El **Informer** mejora la eficiencia, pero todavía compara puntos individuales de la serie mediante atención. Esto puede ser problemático en series temporales, porque un punto aislado puede contener poco significado.

Por ejemplo, dos valores similares pueden pertenecer a momentos diferentes del ciclo. Además, una tendencia fuerte puede ocultar los patrones estacionales.

Autoformer propone una idea diferente: en lugar de comparar puntos individuales, comparar sub-series que se repiten.

Informer: hace la atención más eficiente mediante ProbSparse Attention.

Autoformer: reemplaza la atención por Auto-Correlation, buscando patrones periódicos y sub-series relacionadas.

Autoformer

Idea: Autoformer reemplaza la atención tradicional por un mecanismo basado en autocorrelación y hace que la descomposición de la serie sea parte del modelo.

Series Decomposition Block

- La serie se divide dentro de cada capa en tendencia y estacionalidad:

$$X_{\text{trend}} = \text{MovingAverage}(X)$$

$$X_{\text{seasonal}} = X - X_{\text{trend}}$$

- La media móvil obtiene la tendencia, mientras que la diferencia contiene los patrones estacionales.
- Esta descomposición se realiza progresivamente en cada capa, y no solamente como un preprocesamiento.

Autoformer

Auto-Correlation

- En lugar de comparar puntos individuales mediante atención, Autoformer busca patrones que se repiten a diferentes desfases temporales.
- La autocorrelación se calcula eficientemente mediante FFT.

Informer: busca relaciones entre puntos mediante atención.

Autoformer: busca periodicidades y patrones completos mediante autocorrelación.

Autoformer es especialmente adecuado para series con patrones estacionales o periódicos marcados.

Autoformer

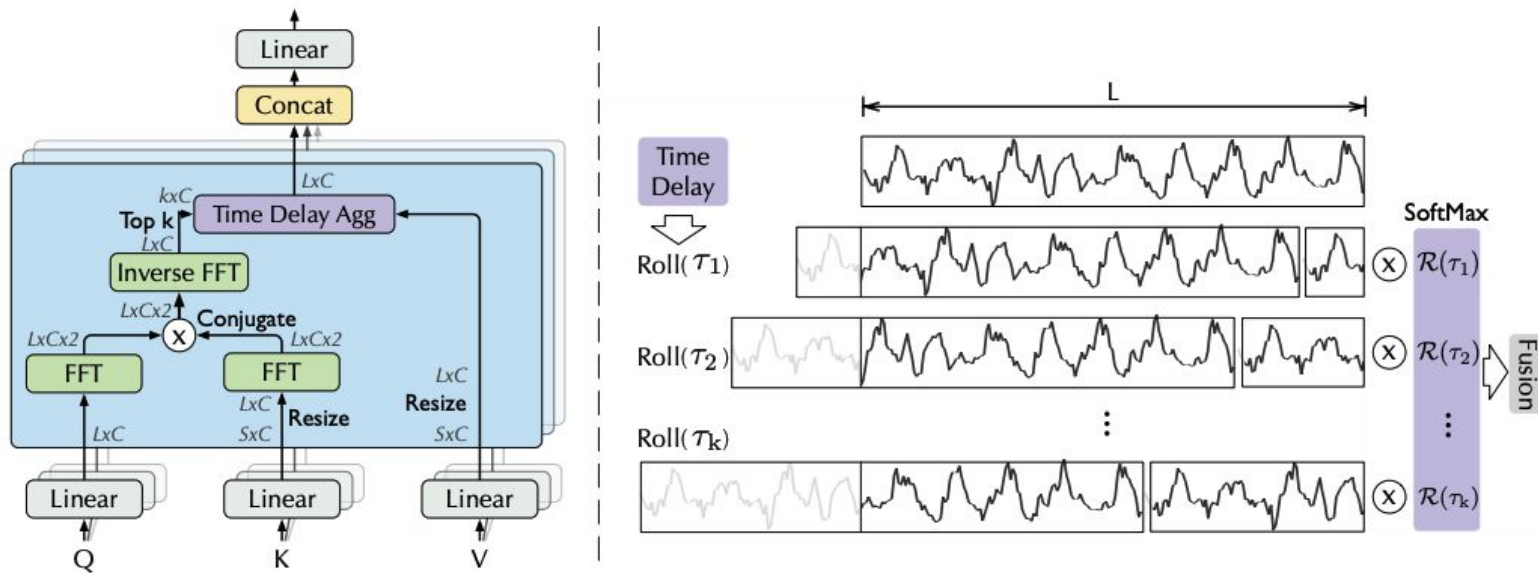


Figure 2: Auto-Correlation (left) and Time Delay Aggregation (right). We utilize the Fast Fourier Transform to calculate the autocorrelation $\mathcal{R}(\tau)$, which reflects the time-delay similarities. Then the similar sub-processes are rolled to the same index based on selected delay τ and aggregated by $\mathcal{R}(\tau)$.

Autoformer

- **Autoformer** rompe con la atención tradicional al incorporar el conocimiento de dominio sobre la periodicidad directamente en su arquitectura. Por eso funciona especialmente bien en series con estacionalidad marcada.
- Informer mejora la eficiencia y Autoformer aprovecha la periodicidad, pero ninguno está diseñado específicamente para integrar múltiples tipos de variables ni para explicar qué factores generan la predicción.

En aplicaciones reales podemos tener:

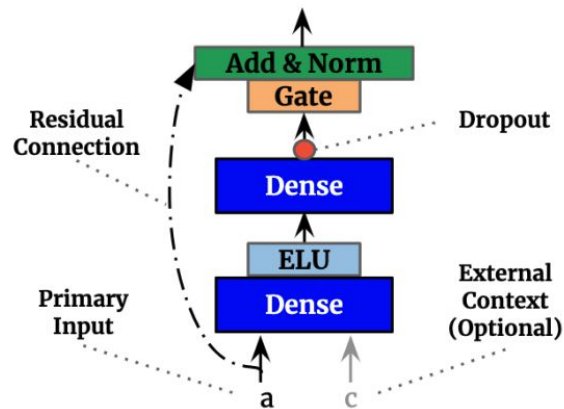
- **Variables conocidas a futuro:** feriados, promociones, calendario, etc.
- **Variables estáticas:** tienda, región, producto, categoría, etc.
- **Etc**

El problema es que estos modelos no tienen un mecanismo especializado para distinguir e integrar estos diferentes tipos de información.

Temporal Fusion Transformer

Idea: pronóstico multi-horizonte, multivariado e interpretable, con heterogeneidad de entradas.

- En vez de optimizar la atención, reorganiza las entradas.
- **Tipos de entrada :**
 - **Covariables estáticas** (id de tienda, región, categoría)
 - **Entradas conocidas a futuro** (día de la semana, feriados, promociones planeadas)
 - **Entradas observadas solo en el pasado** (ventas históricas, tráfico, clima real)
- **Componentes clave:**
 - **Gated Residual Network (GRN):** Permite al modelo apagar componentes innecesarios, funciona bien en datasets chicos y ruidosos.



Gated Residual Network (GRN)

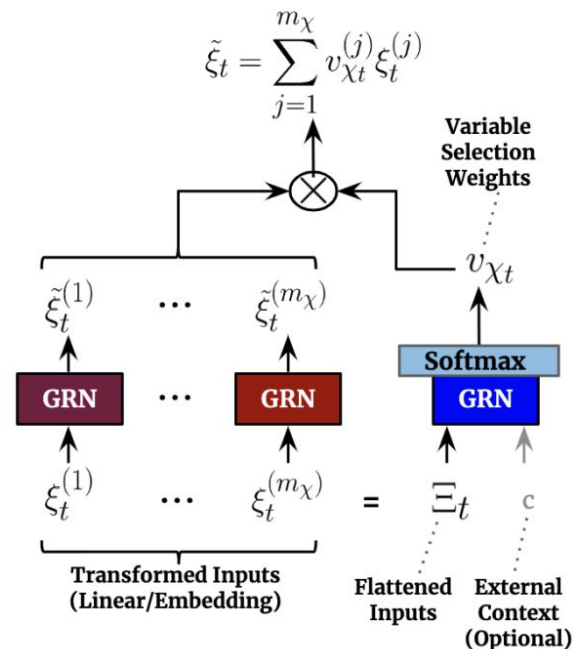
Temporal Fusion Transformer

- **Variable Selection Networks:**

- Selecciona automáticamente las variables más relevantes para la predicción.
- Permite interpretar el modelo, porque podemos observar qué variables recibieron mayor peso.

- **Static Covariate Encoders:**

- Incorporan información que no cambia con el tiempo, por ejemplo:
 - Tienda
 - Región
 - Tipo de producto
- Esta información se utiliza en diferentes partes del modelo para condicionar la predicción según el contexto.



Variable Selection Network

Temporal Fusion Transformer

- **LSTM Encoder-Decoder:**

- El LSTM procesa la secuencia y captura patrones de corto plazo.
- Ayuda al modelo a conocer el orden temporal de los datos, funcionando como una especie de positional encoding aprendido.

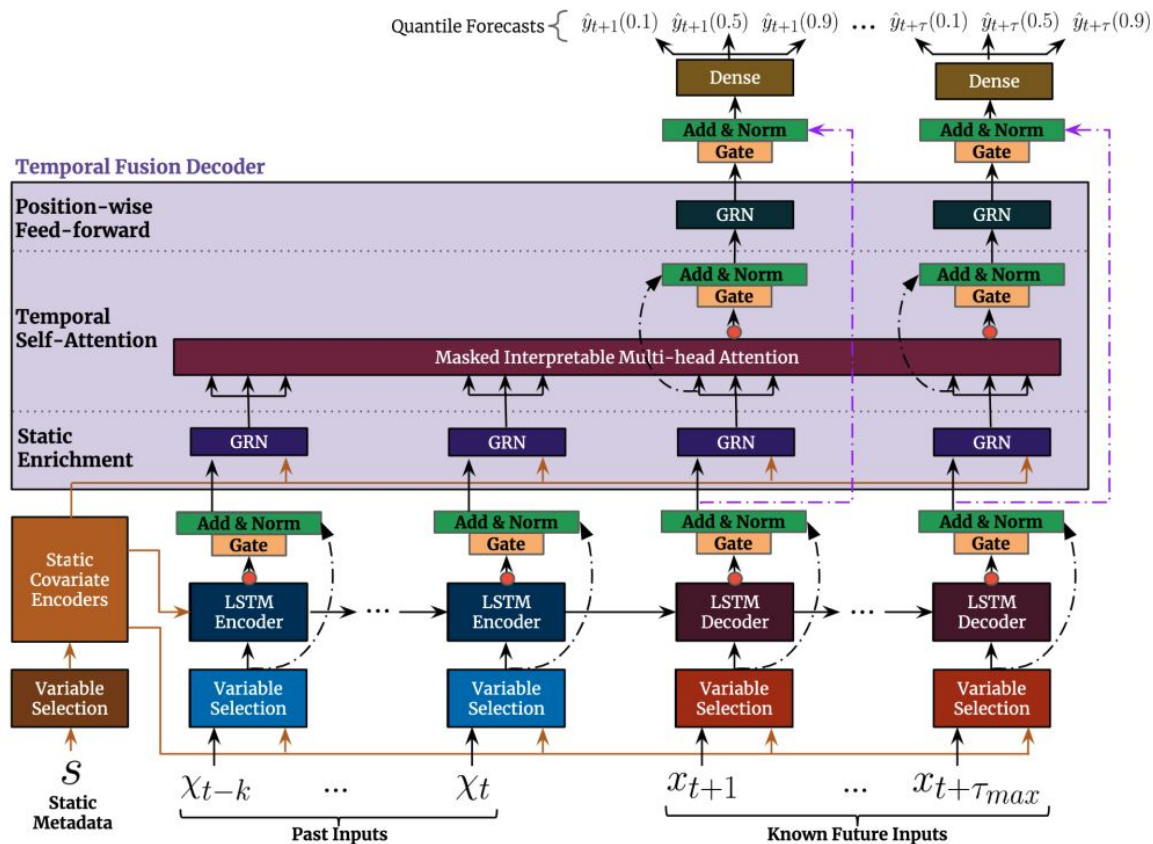
- **Interpretable Multi-Head Attention:**

- Permite identificar qué momentos del pasado son más importantes para realizar una predicción.
- Las diferentes cabezas comparten los mismos valores V , y sus resultados se combinan para facilitar la interpretación de los pesos de atención.
- Permite analizar qué timestamps influyeron más en la predicción.

- **Quantile Loss**

- En lugar de predecir solamente un valor, el modelo puede predecir diferentes cuantiles.
- Esto permite obtener intervalos de predicción: [P10,P90]

Temporal Fusion Transformer



Problemas de DLinear

- DLinear mostró que los Transformers no siempre son necesarios para series temporales.
- Se demostró que un modelo extremadamente simple, basado en descomposición y una capa lineal, podía superar a modelos como **Informer** y **Autoformer** en muchos benchmarks.
- Los Transformers pueden tener dificultades para representar correctamente el orden temporal, mientras que un modelo lineal puede aprovechar directamente la estructura de la secuencia.
- **PatchTST** plantea que el problema no era necesariamente la atención, sino cómo se dividía la serie. **En los Transformers tradicionales: 1 token=1 timestamp.** Un timestamp aislado contiene muy poca información. La atención tiene que aprender los patrones temporales a partir de muchos puntos individuales.

PatchTST (2023)

Idea: el problema no era el Transformer, era cómo se tokenizaba la serie

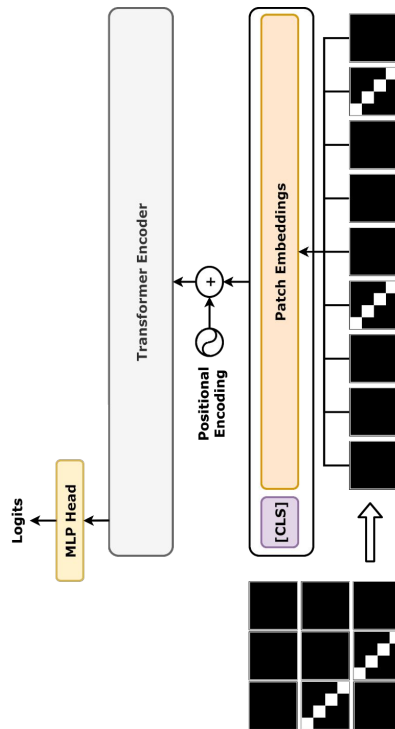
Patching (sub-series como tokens)

- La serie se divide en parches de longitud P usando un stride S :

$$[x_1, \dots, x_P], \quad [x_{1+S}, \dots, x_{P+S}], \quad \dots$$

Cada patch se convierte en un token, de forma similar a cómo funciona un Vision Transformer (ViT) con pequeños bloques de una imagen.

- Más información por token:** cada token representa un pequeño patrón temporal, no un único valor.
- Menor costo computacional:** como la atención depende del número de tokens, reducirlos disminuye considerablemente memoria y cómputo.
- Contextos más largos:** permite procesar ventanas temporales mayores manteniendo un costo razonable.



PatchTST (2023)

Channel Independence

- Cada variable de la serie temporal se procesa por separado, utilizando los mismos pesos del Transformer. Por ejemplo, si tenemos:

Ventas, Precio, Temperatura

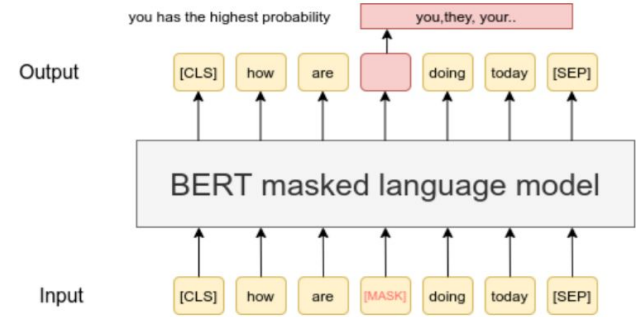
cada una se procesa individualmente.

- **Ventaja:** Evita que el modelo mezcle excesivamente las variables, reduciendo el sobreajuste y mejorando la generalización.
- **Desventaja:** no aprende explícitamente las relaciones entre variables.

PatchTST (2023)

Pre-entrenamiento Self-supervised

- PatchTST también puede aprender sin utilizar directamente las etiquetas de predicción.
- Se ocultan algunos patches de la serie y el modelo debe reconstruirlos.
- Esto permite aprender representaciones generales de las series temporales que posteriormente pueden utilizarse en otros datasets mediante transfer learning.



iTransformer

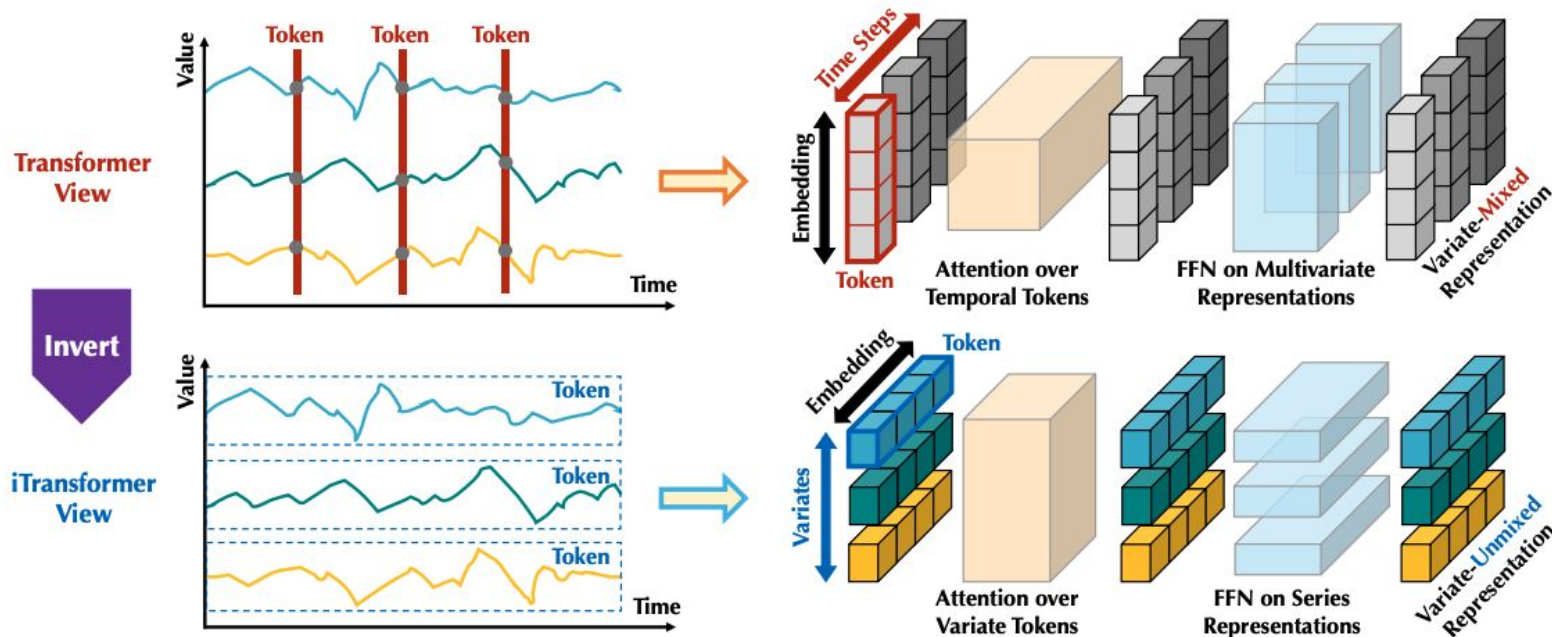


Figure 2: Comparison between the vanilla Transformer (top) and the proposed iTransformer (bottom). Transformer embeds the temporal token, which contains the multivariate representation of each time step. iTransformer embeds each series independently to the variate token, such that the attention module depicts the multivariate correlations and the feed-forward network encodes series representations.

Resumen

Modelo	Limitación que viene del anterior modelo	Cómo lo maneja
Vanilla Transformer		Self-attention completa entre timestamps
Informer	Impide horizontes largos. Inferencia paso a paso	ProbSparse, self-attention distilling, decoder generativo
Autoformer	Comparar puntos aislados no captura patrones; la tendencia tapa el ciclo	Auto-correlación por FFT y descomposición dentro de cada capa
Paper de DLinear	Bajo rendimiento de Informer y Autoformer por el manejo de tokens.	
TFT	El problema real tiene covariables estáticas/futuras, riesgo e interpretabilidad	VSN, GRN/GLU, atención interpretable y pinball loss (cuantiles)
PatchTST	La falla no era la atención sino la tokenización; el contexto era muy corto	Patching, channel independence, RevIN, pre-entrenamiento enmascarado

Transformers

[LINK](#)



Ejercicio, Transformers

[LINK](#)



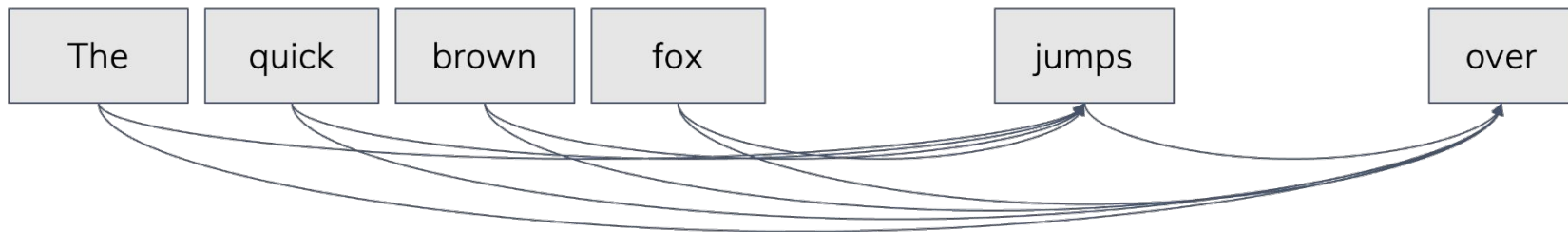
Alternativas al Attention

State Space Models

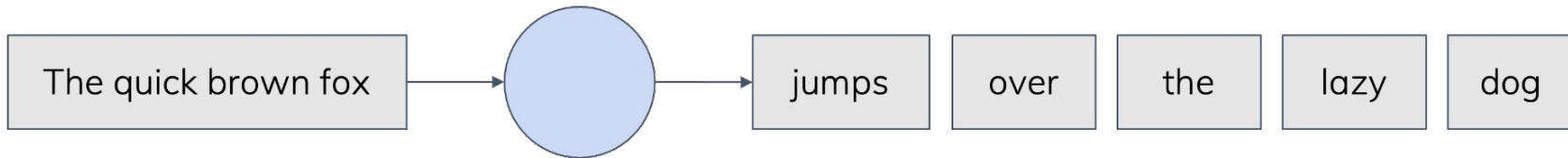
- Son una clase de algoritmos de ML utilizados para realizar predicciones sobre sistemas dinámicos, modelando cómo evoluciona su estado interno a lo largo del tiempo mediante ecuaciones diferenciales.
- Históricamente, los SSM se han utilizado en la ingeniería de sistemas de control, pero actualmente constituyen una plataforma matemática muy flexible que puede adaptarse a la mayoría de las tareas de modelado de secuencias.
- Mamba, una arquitectura de red neuronal basada en SSM, compite con los Transformers en cuanto a rendimiento en tareas de modelado del lenguaje.

State Space Models

Transformers generate **quadratically** by attending to every past token



In SSMs, tokens stream in, update the model's state and are discarded



State is highly compressed, enabling generation in **linear time**

Mamba

- En lugar de comparar todos los puntos con todos (atención), mantiene un estado oculto que se actualiza paso a paso, como un sistema dinámico.
- Mamba decide dinámicamente qué recordar y qué olvidar (por ejemplo, ignorar ruido y retener un cambio de régimen).
- **Coste computacional:** lineal, $O(L)$, frente a $O(L^2)$ del Transformer. Con L = longitud de la serie. Por eso es una buena opción para ventanas históricas muy largas.

TS Mamba

Lo nuevo:

- **Parching:** Divide la serie en segmentos (patches) en vez de tratar cada punto suelto, reduce longitud y captura patrones locales.
- **Encoders bidireccionales:** Un Mamba hacia adelante y otro hacia atrás. La predicción no es causal estricta, así que aprovechar el contexto en ambos sentidos mejora la representación.
- **Channel Independence:** El backbone procesa cada variable por separado.

Objetivo: ser un foundation model, predecir en zero-shot sobre series nunca vistas, con mucho menos dato de entrenamiento

TSMamba

Specialized Time Series vs Foundation Models

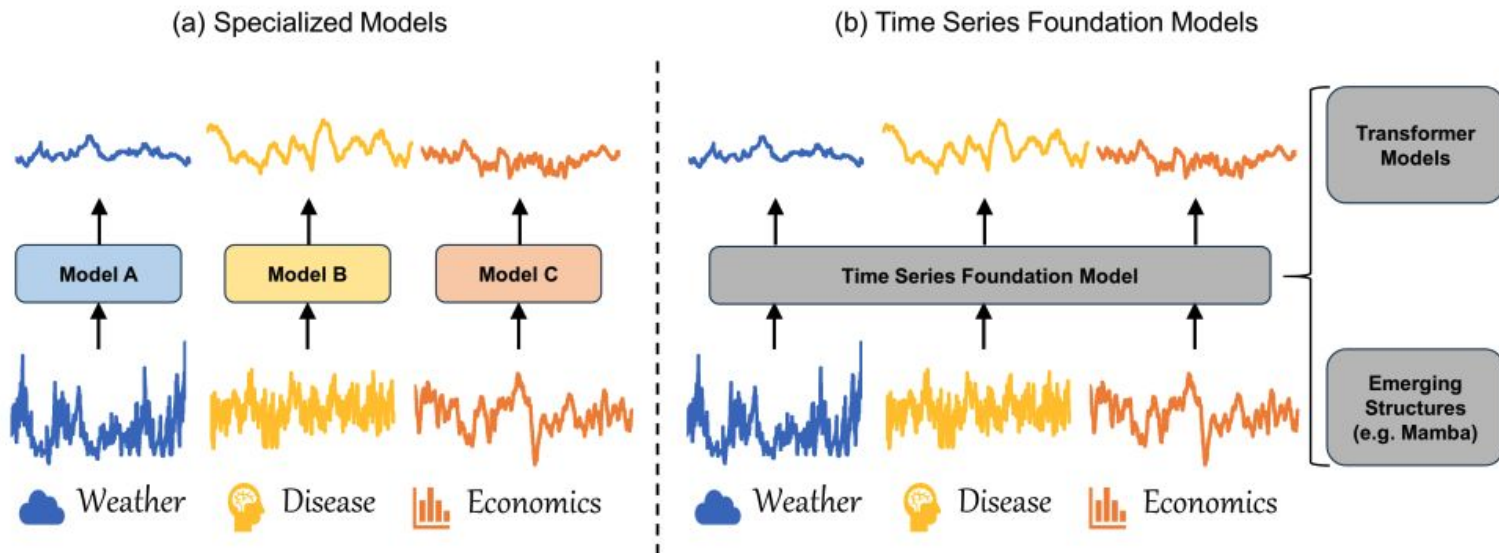


Figure 1: Comparison of specialized time series models and foundation models: (a) Specialized models are trained separately for specific tasks using relevant datasets. These models lack the ability to generalize across different domains and frequencies. (b) Time series foundation models, trained on large datasets, generalize well across a wide range of scenarios and tasks.

TSMamba

Arquitectura

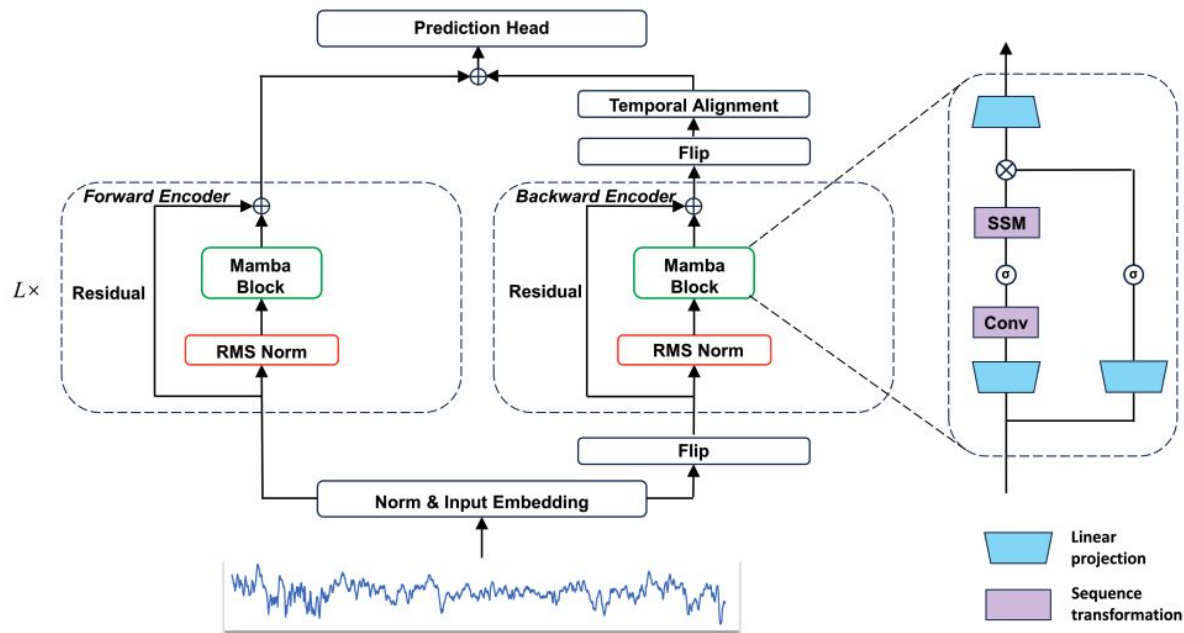


Figure 2: TSMamba Architecture: The input time series are preprocessed and then fed into the forward and backward encoder to extract internal dependencies. The representations are combined and subsequently mapped to forecasts by the prediction head.

TSMixer

TSMixer: An All-MLP Architecture for Time Series Forecasting

Si-An Chen

*National Taiwan University
Google Cloud AI Research*

d09922007@ntu.edu.tw

Chun-Liang Li

Google Cloud AI Research

chunliang@google.com

Nathanael C. Yoder

Google Cloud AI Research

nyoder@google.com

Sercan Ö. Arık

Google Cloud AI Research

soarik@google.com

Tomas Pfister

Google Cloud AI Research

tpfister@google.com

Reviewed on OpenReview: <https://openreview.net/forum?id=wbpaTuXgm0>

Abstract

Real-world time-series datasets are often multivariate with complex dynamics. To capture this complexity, high capacity architectures like recurrent- or attention-based sequential deep learning models have become popular. However, recent work demonstrates that simple univariate linear models can outperform such deep learning models on several commonly used academic benchmarks. Extending them, in this paper, we investigate the capabilities of linear models for time-series forecasting and present Time-Series Mixer (TSMixer), a novel architecture designed by stacking multi-layer perceptrons (MLPs). TSMixer is based on mixing operations along both the time and feature dimensions to extract information efficiently. On popular academic benchmarks, the simple-to-implement TSMixer is comparable to specialized state-of-the-art models that leverage the inductive biases of specific benchmarks. On the challenging and large scale M5 benchmark, a real-world retail dataset, TSMixer demonstrates superior performance compared to the state-of-the-art alternatives. Our results underline the importance of efficiently utilizing cross-variate and auxiliary information for improving the performance of time series forecasting. We present various analyses to shed light into the capabilities of TSMixer. The design paradigms utilized in TSMixer are expected to open new horizons for deep learning-based time series forecasting. The implementation is available at: <https://github.com/google-research/google-research/tree/master/tsmixer>.

TS Mixer

Idea: mezclar información a lo largo del tiempo y entre variables con MLPs

- Modelos lineales simples (DLinear) superaban a Transformers complejos en benchmarks estándar. Quizá la atención es innecesaria.
- **Arquitectura.**
 - Sin atención, sin recurrencia.
 - Solo MLPs.
 - La serie multivariante es una matriz [tiempo × variables] y se alternan dos operaciones:
 - **Time-mixing:** un MLP aplicado a lo largo del eje temporal, compartido entre todas las variables, captura patrones temporales (tendencia, estacionalidad).
 - **Feature-mixing:** un MLP aplicado a lo largo del eje de variables, compartido entre todos los instantes, captura correlaciones entre series (ej: temperatura con demanda eléctrica).

TS Mixer

- Se apilan varios bloques con residuales y normalización 2D (sobre ambos ejes a la vez), clave para estabilizar escalas.
- **Ventajas:** extremadamente rápido, pocos parámetros, muy buen rendimiento (supera a PatchTST, Autoformer y otros)
- **Limitación estructural:** el MLP temporal tiene tamaño fijo, así que la longitud de la ventana de entrada queda fijada en el diseño. No hay memoria de estado ni extrapolación a secuencias más largas.

TSMixer

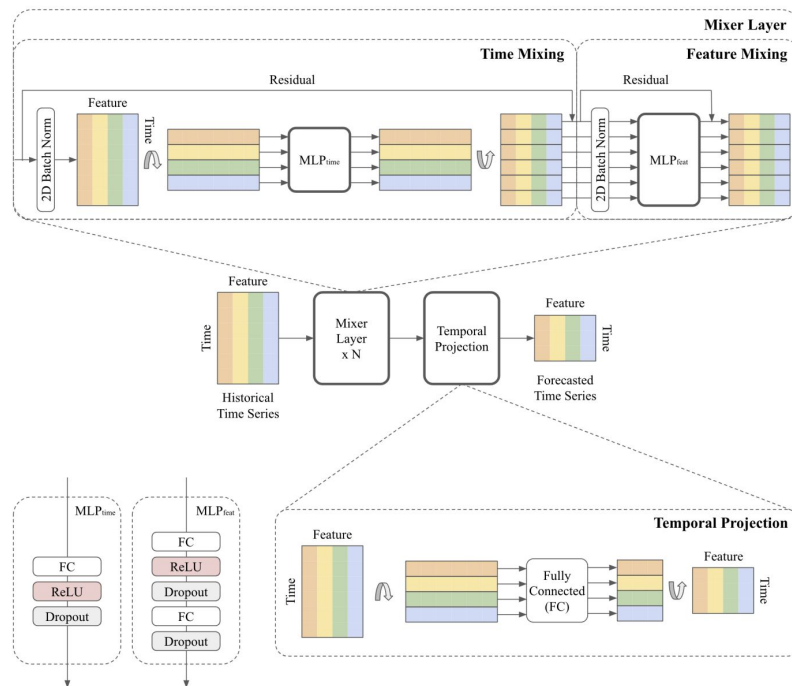


Figure 1: TSMixer for multivariate time series forecasting. The columns of the inputs means different features/variables and the rows are time steps. The fully-connected operations are row-wise. TSMixer contains interleaving time-mixing and feature-mixing MLPs to aggregate information. The number of mixer layer is denoted as N . The time-mixing MLPs are shared across all features and the feature-mixing MLPs are shared across all of the time steps. The design allow TSMixer to automatically adapt the use of both temporal and cross-variate information with limited number of parameters for superior generalization. The extension with auxiliary information is also explored in this paper.

TSMixer

[LINK](#)

